Task-4:- Documentation(Deploying Frontend Applications on EC2, ECS, and S3)

Project: Deploy Frontend Applications on AWS (EC2, ECS, and S3)

Objective

Deploy three frontend applications:

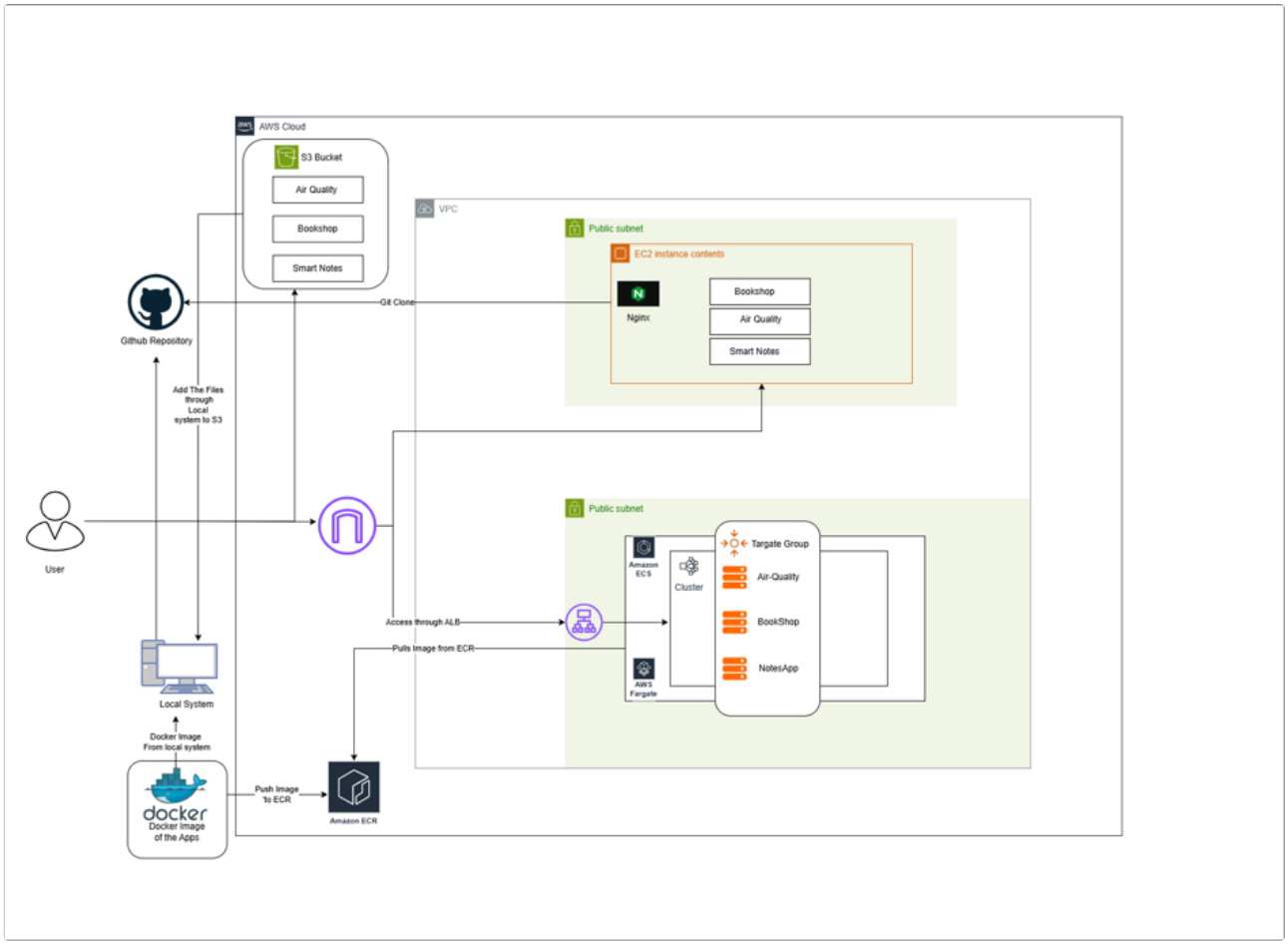
- **Air Quality**
- **Bookshop**
- **NotesApp**

Using three different AWS services:

- Amazon EC2
- Amazon ECS (Fargate + ALB)
- Amazon S3 (Static Website Hosting)

The objective is to deploy and verify all three applications on each service and compare deployment strategies.

A. Architecture and Infrastructure



A.1 Infrastructure Overview

Core Components

- EC2 Instance (Nginx)
- Docker Engine
- Amazon ECR
- Amazon ECS Fargate
- Amazon S3
- Security Groups
- Default VPC Networking
- GitHub Repository

Application Architecture and Data Flow

1. DevOps engineer retrieves source code.
2. Applications are deployed to EC2 via Nginx.

3. Docker images are built and pushed to ECR.
4. ECS services pull images and run containers.
5. Static builds are uploaded to S3.
6. End users access applications via public endpoints.

A.2 Environments

Environment	Purpose
Development	Local testing
Staging	Deployment validation
Production	Live frontend hosting

A.3 Tools and Technologies

Tool	Purpose

Tool	Purpose
Nginx	Static web server
Docker	Containerization
Amazon ECR	Image registry
Amazon ECS	Container orchestration
AWS Fargate	Serverless containers
Amazon S3	Static hosting
GitHub	Source repository
AWS CLI	Cloud interaction
Security Groups	Network control

A.4 User Request Flow

1. User accesses public URL.
2. Request reaches hosting platform (EC2/ECS/S3).
3. Static assets served via Nginx or S3.
4. Browser renders frontend application.
5. Health checks monitored by ECS where applicable.

SECTION 1: Deployment on Amazon EC2

1. Launch EC2 Instance

Launched an Ubuntu EC2 instance with:

- Instance Type: t2.micro
- Public IP assigned
- Security Group Inbound Rules:
  - HTTP (80) – Anywhere (0.0.0.0/0)
  - Custom TCP Ports:
    - 8080 (Air Quality)
    - 8081 (Bookshop)
    - 8082 (NotesApp)
  - SSH (22)

---

2. Install and Configure Nginx

Installed Nginx and verified service status.

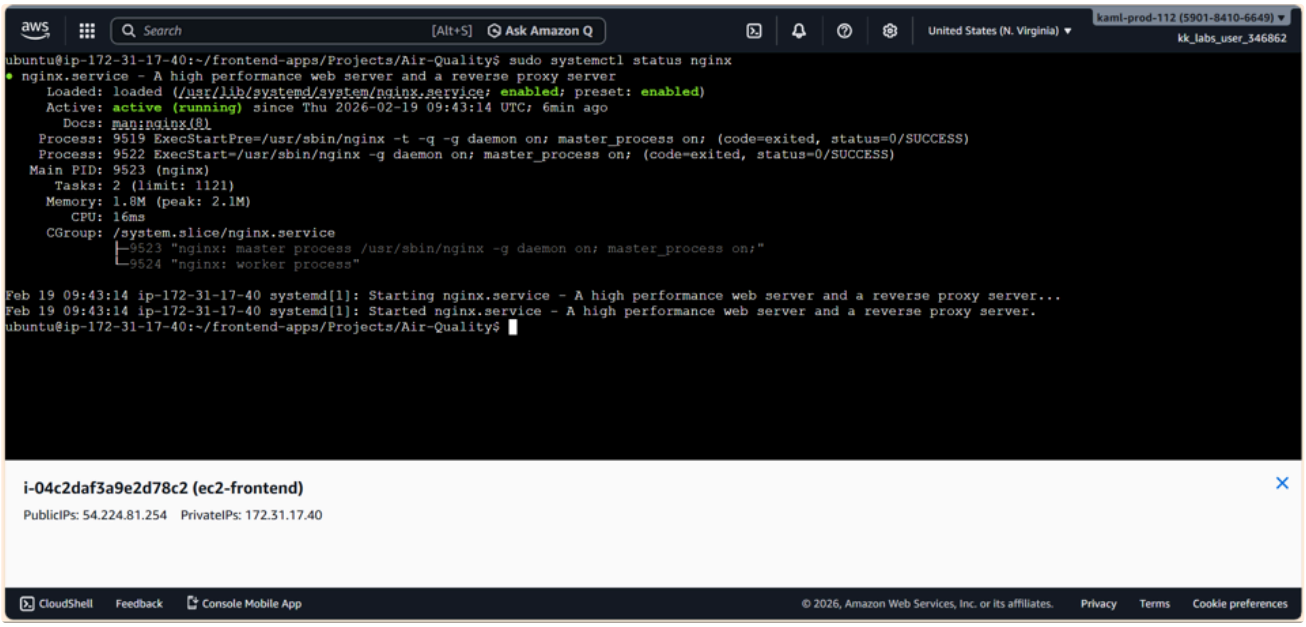
Command:

```
sudo systemctl status nginx

```

Status:

- Active: running



3. Verify Listening Ports

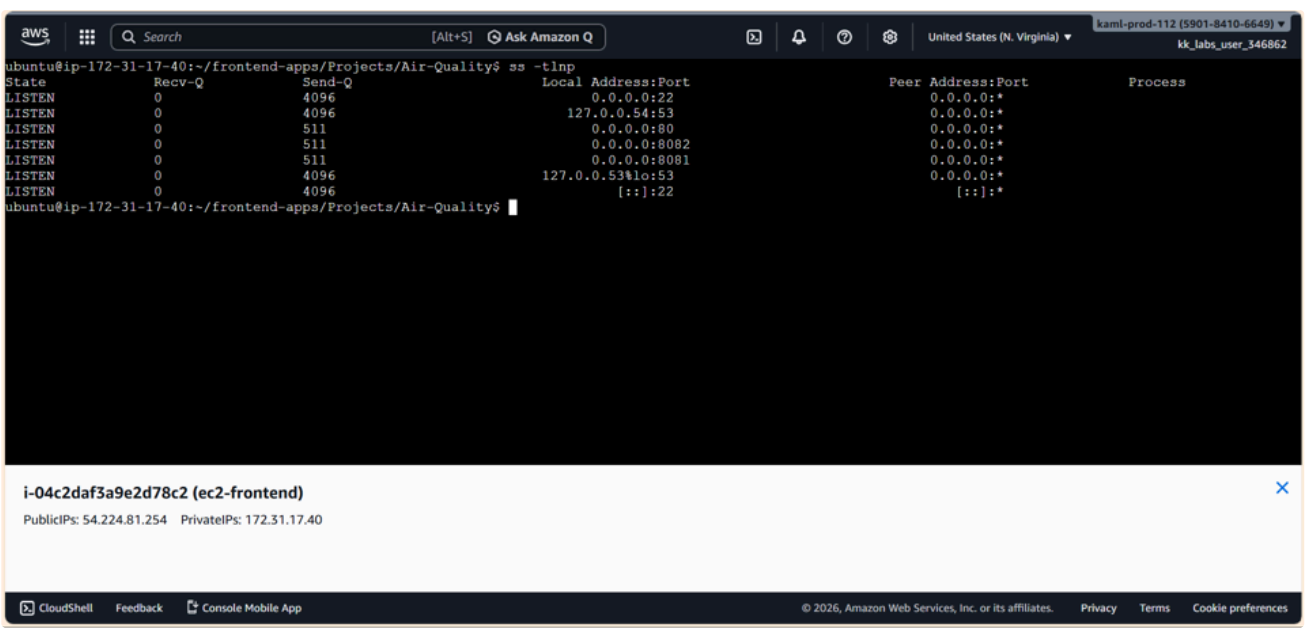
Verified application ports using:

```
ss -tlnp

```

Observed listening ports:

- 80
- 81
- 82



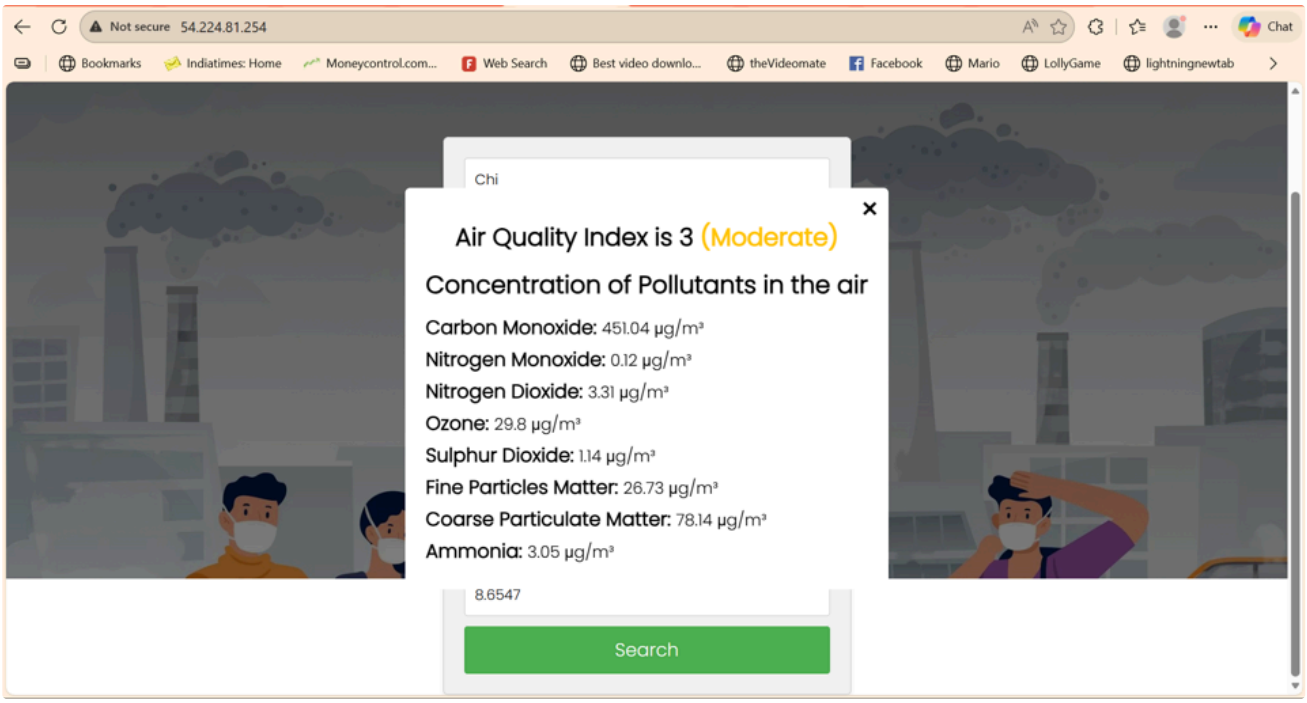
4. Application Verification on EC2

Air Quality

Accessed via:

```
http://15.206.169.206:3001

```

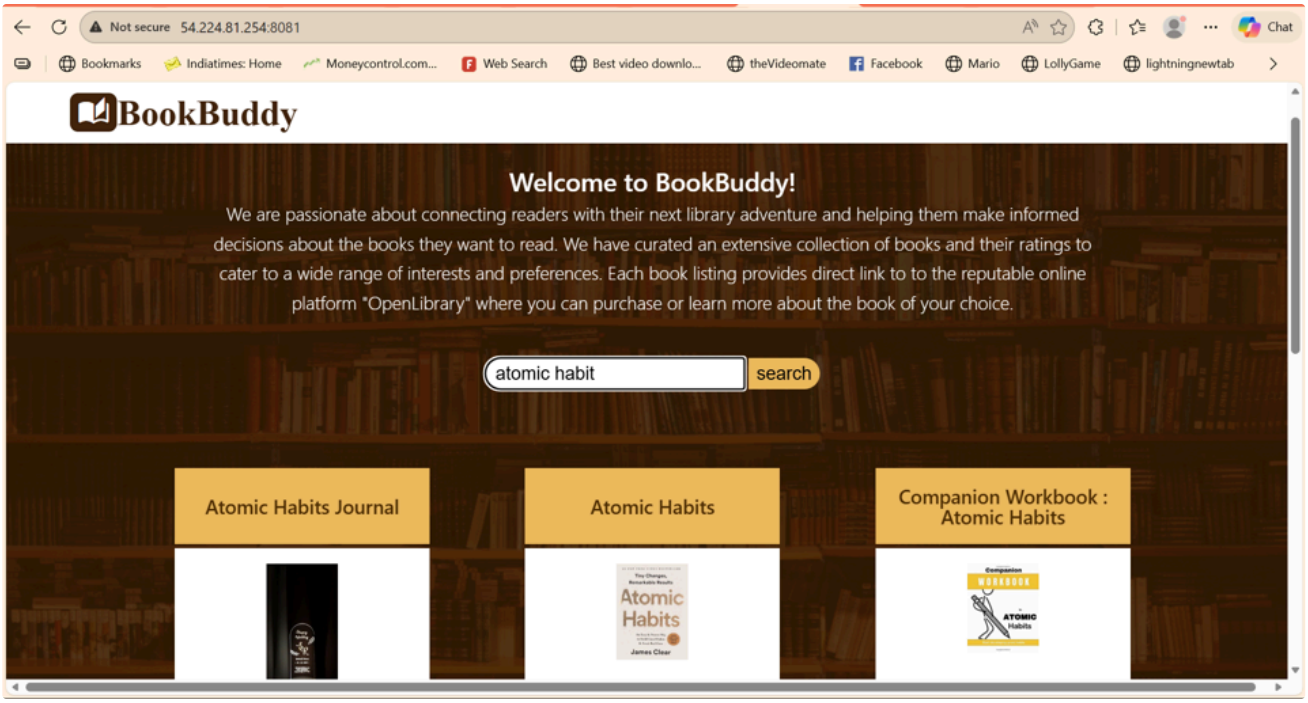


Bookshop

Accessed via:

```
http://15.206.169.206:3002

```

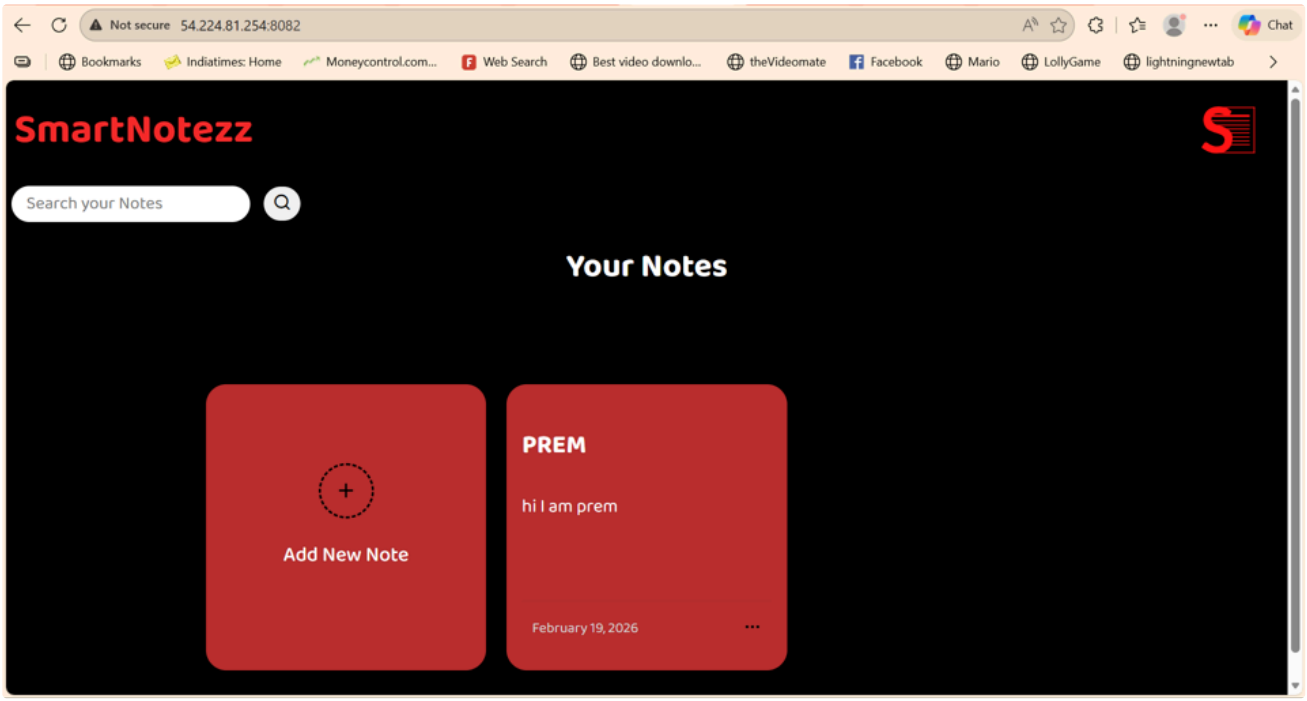


NotesApp

Accessed via:

```
http://15.206.169.206:3003

```



SECTION 2: Deployment on Amazon ECS (Fargate)

1. Create ECS Cluster

Created an ECS Cluster using:

- Launch Type: Fargate
- Status: Active

---

2. Create Task Definitions

Created separate task definitions for:

- Air Quality
- Bookshop
- NotesApp

Configuration:

- Launch Type: Fargate
- Network Mode: awsvpc
- CPU: 0.5 vCPU
- Memory: 1 GB
- Container Port: 80

---

3. Create ECS Services

Created services for each application:

- Desired Tasks: 1
- Running Tasks: 1
- Target Type: IP

---

4. Create Target Groups

Created Target Groups with:

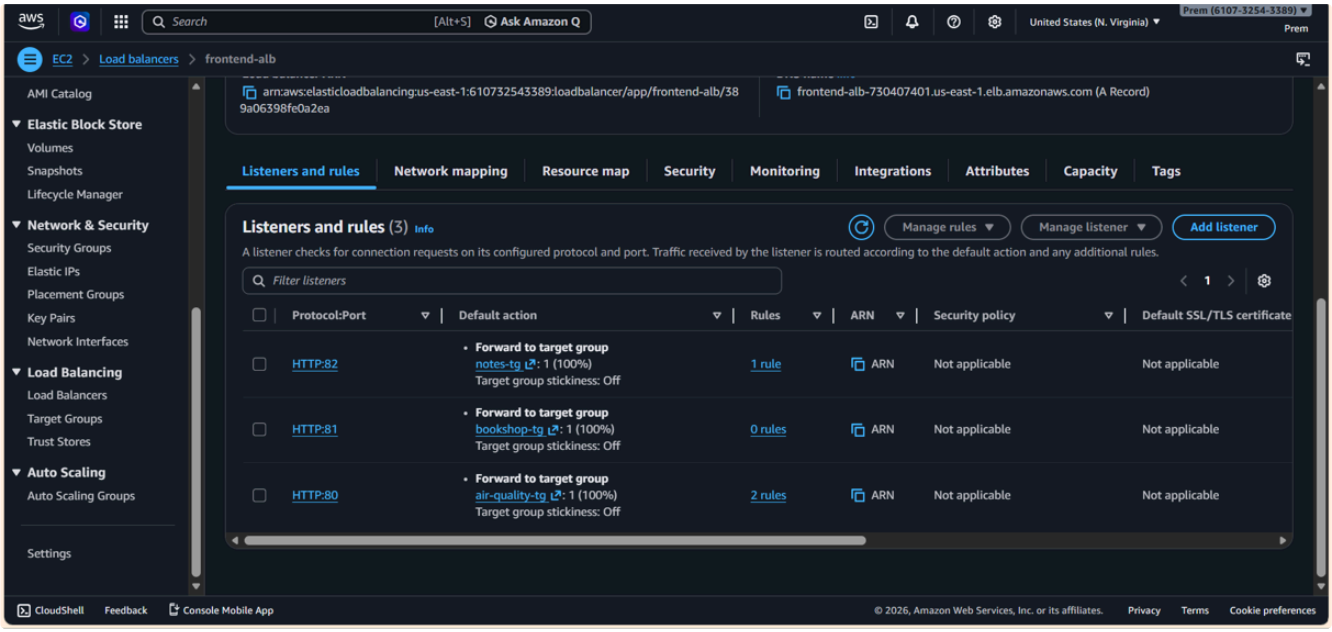
- Protocol: HTTP
- Port: 80
- Health Check Path: /

---

5. Create Application Load Balancer (ALB)

Created an Internet-facing ALB:

- Listener Ports:
  - 80 → Air Quality
  - 81 → Bookshop
  - 82 → NotesApp
- Forwarded to respective Target Groups
- Status: Active

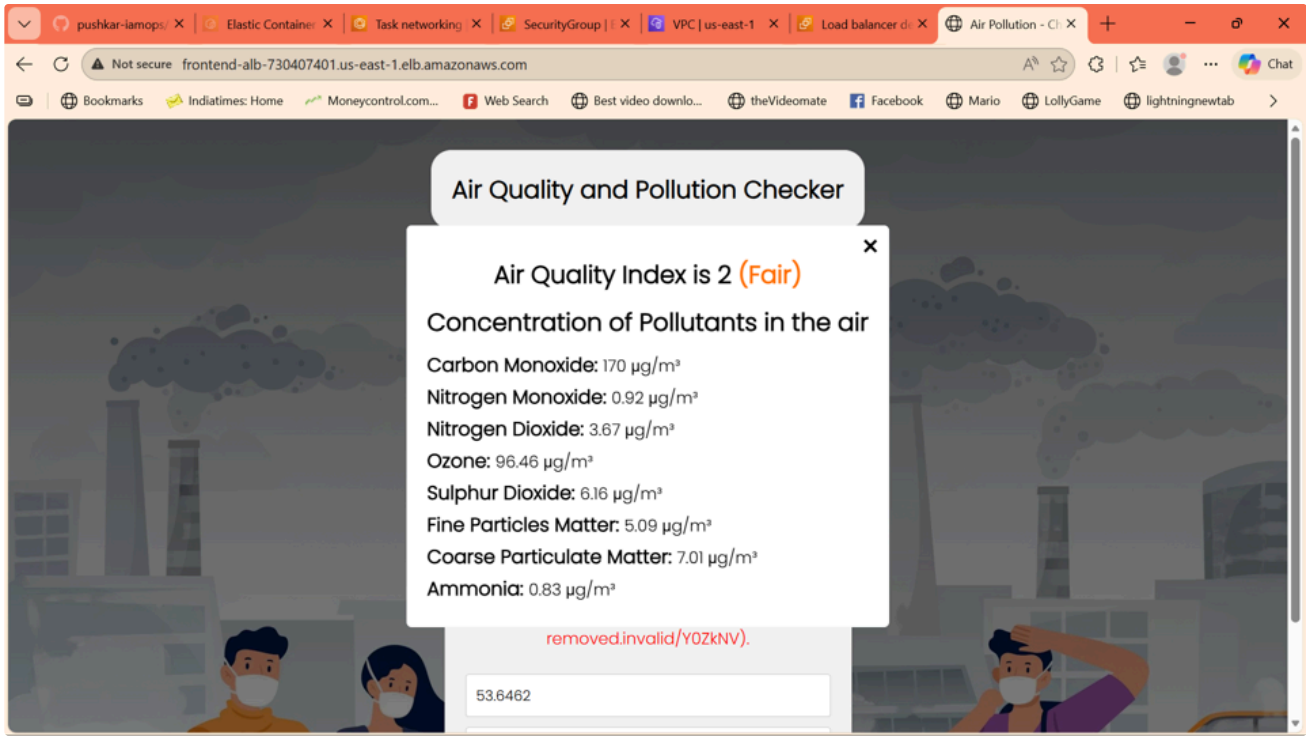


---

6. Application Verification via ALB

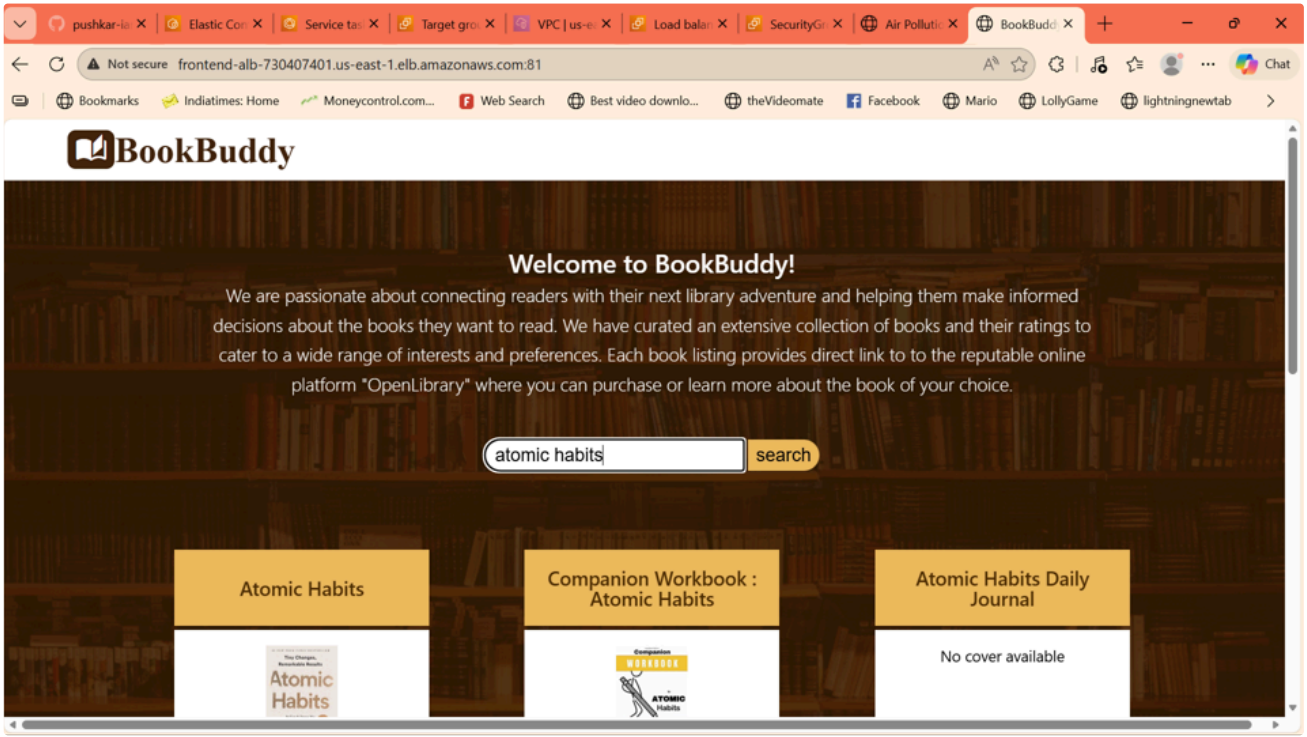
Air Quality (ECS)

```
http://frontend-alb-869375343.ap-south-1.elb.amazonaws.com/
```



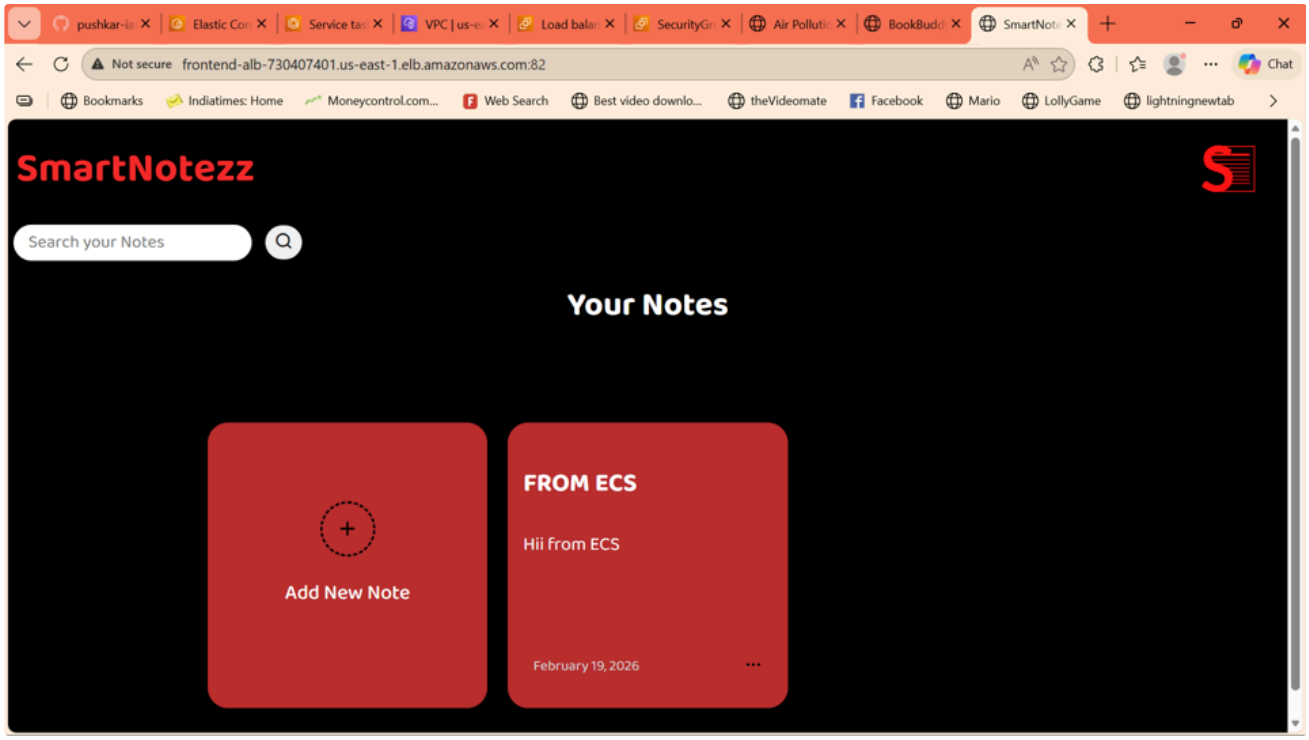
Bookshop (ECS)

```
http://frontend-alb-869375343.ap-south-1.elb.amazonaws.com:81/
```



NotesApp (ECS)

```
http://frontend-alb-869375343.ap-south-1.elb.amazonaws.com:82/
```



Networking Configuration (ECS)

- ALB Security Group:
  - Inbound: HTTP (80, 81, 82) from Anywhere
- ECS Task Security Group:
  - Inbound: Port 80 from ALB Security Group
- Network Mode: awsvpc
- Public IP assigned to Fargate tasks

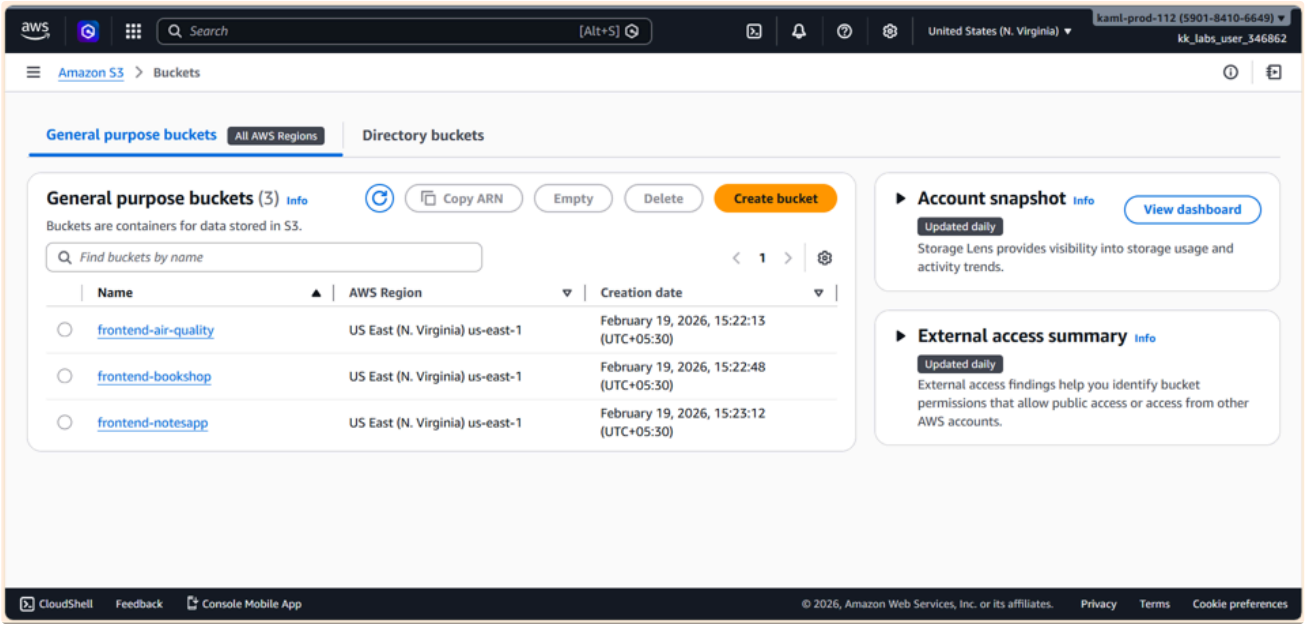
SECTION 3: Deployment on Amazon S3 (Static Website Hosting)

1. Create S3 Buckets

Created three buckets:

- frontend-air-quality
- frontend-bookshop
- frontend-notesapp

Region: us-east-1



---

2. Enable Static Website Hosting

For each bucket:

- Enabled Static Website Hosting
- Index document: index.html
- Public Access enabled via bucket policy

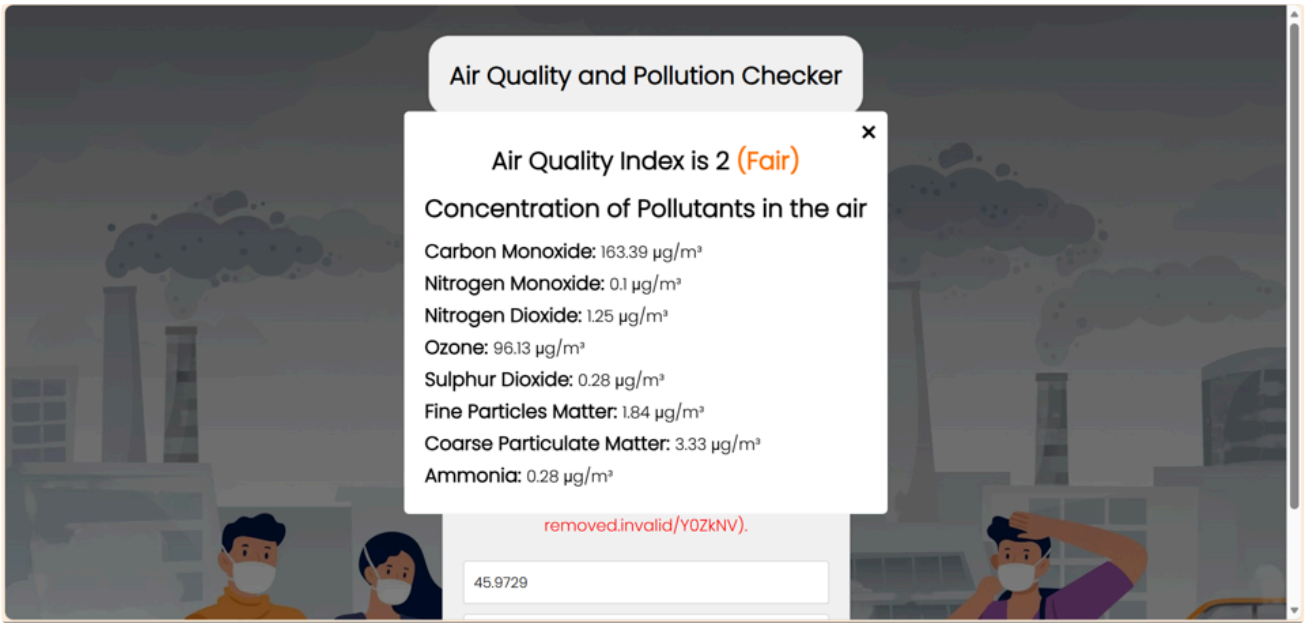
---

3. Application Verification via S3

Air Quality (S3)

Accessed via:

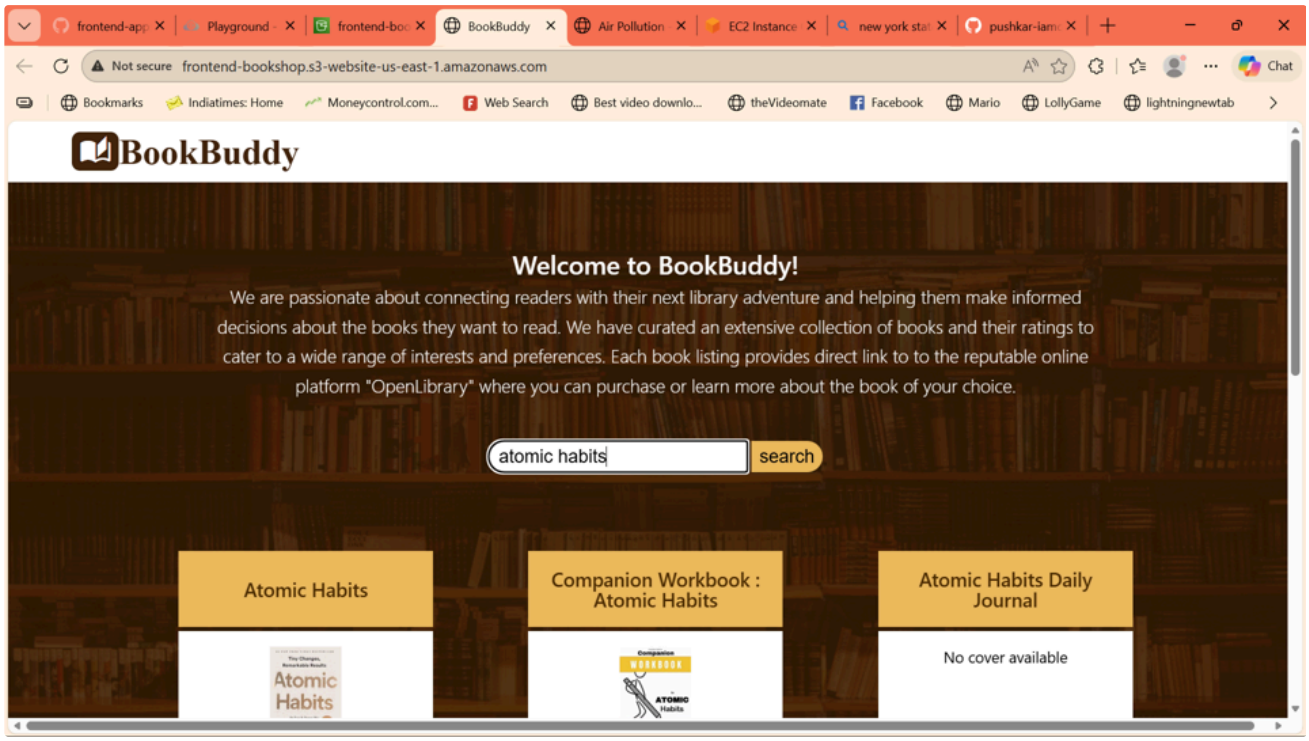
```
http://frontend-air-quality.s3-website-us-east-1.amazonaws.com
```



Bookshop (S3)

Accessed via:

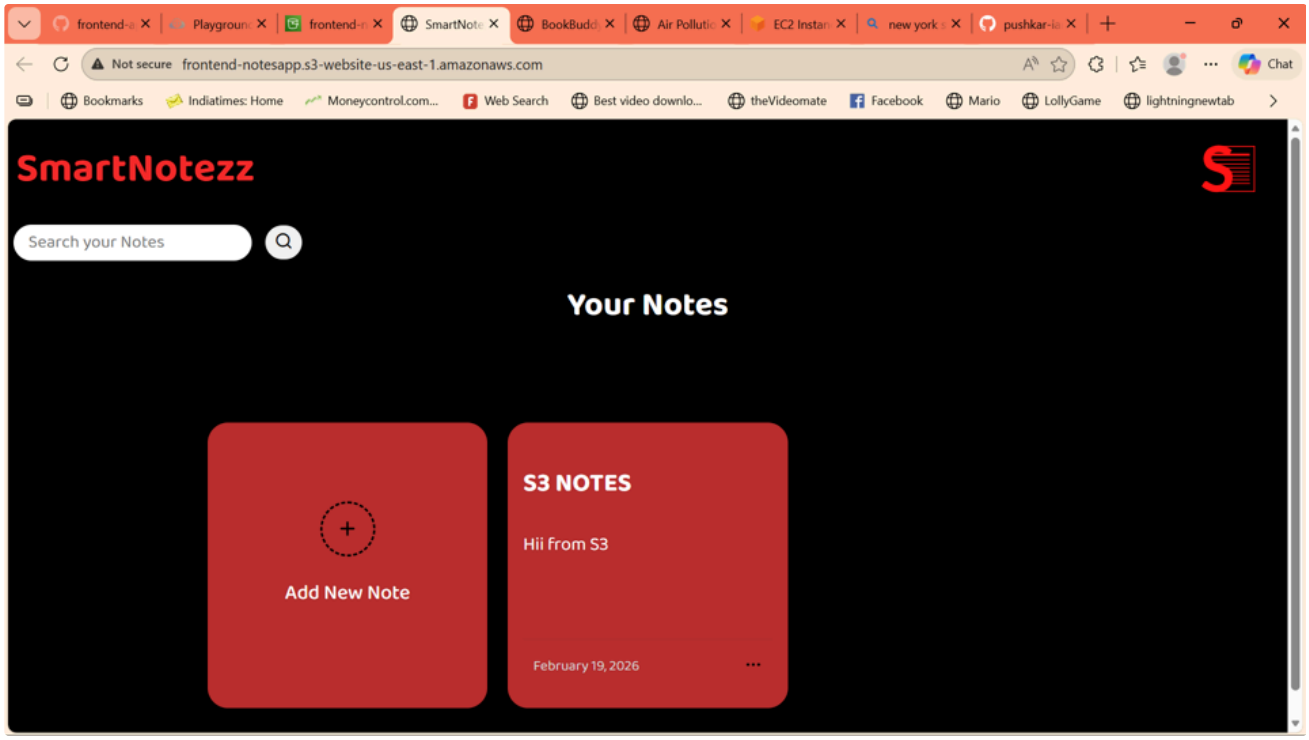
```
http://frontend-bookshop.s3-website-us-east-1.amazonaws.com
```



NotesApp (S3)

Accessed via:

```
http://frontend-notesapp.s3-website-us-east-1.amazonaws.com
```



Final Architecture Flow

EC2 Architecture

User → EC2 Public IP → Nginx → Static Files

ECS Architecture

User → Application Load Balancer

↓

Target Group

↓

ECS Fargate Service (Running Tasks)

↓

Containerized Frontend Application

S3 Architecture

User → S3 Static Website Endpoint

↓

Public S3 Bucket

↓

HTML/CSS/JS Files

Conclusion

All three frontend applications were successfully:

- Deployed on Amazon EC2 using Nginx
- Containerized and deployed using Amazon ECS (Fargate)
- Hosted as static websites using Amazon S3

Each deployment method was verified successfully through public endpoints.

This consolidated deployment demonstrates:

- Infrastructure-based hosting (EC2)
- Containerized serverless hosting (ECS Fargate)
- Fully managed static hosting (S3)

The environment is fully functional and ready for evaluation and comparison for standard frontend deployment selection.